

Rechnerarchitektur

1. Übung

Aufgabe 1

Im Laufe des Semesters werden wir verschiedene Programme auf den Poolrechnern verwenden. Um zu überprüfen, ob Sie korrekt installiert sind, führen Sie folgende Programme aus:

- GNU C Compiler für x86 Befehlssatz: `gcc`
- Objdump für x86 Befehlssatz: `objdump`
- GNU C Compiler für RISC-V Befehlssatz:
`/usr/local/srinkel/riscv-toolchain/bin/riscv64-unknown-elf-gcc`
- Linux Performance Analyse Tool Perf: `perf`
- Ripes RISC-V Simulator: `/usr/local/srinkel/ripes`

Den Ripes Simulator können Sie auch kostenlos von <https://github.com/mortbopet/Ripes/releases> auf Ihrem privaten Rechner installieren. (Hinweis: Ripes funktioniert nicht auf ARM Prozessoren)

Aufgabe 2

In dieser Übungsaufgabe werden Sie sich mit dem GCC C Compiler vertraut machen.

- Erstellen Sie ein *Hello C* Programm, wie unten abgebildet.
- Kompilieren Sie das Programm mit `gcc -o ausgabe eingabe`, wobei *ausgabe* der Name des Programms ist und *eingabe* der Name der C Quellcode Datei.
- Starten Sie ihr Programm und überprüfen Sie, ob es korrekt funktioniert.

```
1 #include <stdio.h>
2
3 int main() {
4
5     printf("Hello C\n");
6
7 }
```

Aufgabe 3

Bestimmen Sie mit Hilfe von C-Anweisungen die Größe des Datentyps `int` und geben Sie die Größe auf der Konsole aus.

Aufgabe 4

Erstellen Sie ein C Programm, dass eine Integer-Variable deklariert und initialisiert. Geben Sie auf der Konsole die Speicheradresse an, wo diese Variable liegt. Führen Sie ihr Programm mehrfach aus, was beobachten Sie?

Aufgabe 5

Erstellen Sie einen zusammengesetzten Datentyp (struct). Der Datentyp soll als erstes Element einen char und als zweites einen int enthalten. Nutzen Sie dazu folgende Definition:

```
1 struct composite {
2     char aChar;
3     int d;
4 };
```

Schreiben Sie ein Programm, was die Größe des Structs in Bytes ausgibt.

Welche Größe erwarten Sie und was wird ausgegeben? Erklären Sie den Unterschied?

Weisen Sie dem Struct folgende Werte zu:

```
1 struct composite cmp;
2 cmp.aChar = '/'; // = 47 (dec) = 0x2F
3 cmp.d = 0xdeadbeef;
```

Geben Sie nun alle Bytes des Structs aus. Zur Ausgabe können Sie folgenden Code verwenden:

```
1 unsigned char *start = (unsigned char *) &cmp;
2
3 for(unsigned char offset = 0; offset < size; offset++){
4     printf("Byte: %2d --- at: %p --- %02X\n", offset, start+offset, start[offset]);
5 }
6 printf("\n");
```

Was beobachten Sie?

Aufgabe 6

Untersuchen Sie das Memory-Alignment des folgenden Structs:

```
1 struct composite {
2     char aChar;
3     int anInt;
4     double aFloat;
5 };
```

Nutzen Sie dazu das Macro `offsetof(...)` wie folgt:

```
1 printf("offsets: aChar = %ld, anInt = %ld, aFloat = %ld\n",
2     offsetof(struct composite, aChar),
3     offsetof(struct composite, anInt),
4     offsetof(struct composite, aFloat));
```

Dazu müssen Sie folgenden Header einbinden: `#include <stddef.h>`.

Wie können Sie das Alignment einer Variablen zur Laufzeit prüfen? Warum kann dies erforderlich sein?

Aufgabe 7

Die nächste Aufgabe funktioniert nur mit dem GCC Compiler. Man kann das Memory-Layout eines structs packen, z.B., um Speicherplatz zu sparen:

```
1 struct composite {
2     char aChar;
3     int anInt;
4     double aFloat;
5 }__attribute__((__packed__)); //works only with gcc
```

Beschreiben Sie das Speicherabbild der Struktur, indem Sie die Methoden der vorherigen Aufgaben nutzen. Erklären Sie die Unterschiede in den Offsets.

Aufgabe 8

In dieser Aufgabe untersuchen wir den Assembler-Code eines minimalen C Programms:

```
1 int main() {  
2     // a very simple program to be displayed as assembler code  
3  
4     int a = 1;  
5     int b = 2;  
6  
7     int c = a + b;  
8  
9     // compile with: gcc -S -fverbose-asm -g -O0 09_assembly.c  
10    // or use objdump with compiled binary file  
11 }
```

Kompilieren Sie das Programm mit den angegebenen Befehlen und sehen Sie sich die Ausgaben an. Finden Sie im Assembler-Code die Zeile, an der die Variablen a und b addiert werden. Wie viele Befehle benötigt die CPU dazu?